

0.33.2. Écrire soi-même sa fonction — doublé, deux positions

UNE fonction, DEUX lettres : `mon_deplace_x` sert pour le A (x) et pour le B (xb).

1. Ce qu'il faut comprendre

C'est le 0.33 et le 0.33.1 réunis : quatre appels à la même fonction.

Écrivez une fois, servez autant qu'on veut : la fonction ne sait rien du A ni du B ; elle bouge ce qu'on lui donne, et rend la colonne. `x =` et `xb =` rangent chaque résultat dans la bonne variable.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1649 octets** de cartouche.

1. UNE VARIABLE — X

```
uint8_t x = 0;          // le A
```

2. UNE VARIABLE — XB

```
uint8_t xb = 0;         // le B

// Notre fonction, la même qu'au 0.33.
```

3. UNE FONCTION — MON_DEPLACE_X()

```
uint8_t mon_deplace_x(uint8_t colonne, uint8_t ligne, uint8_t tuile, uint8_t pas) {
    uint8_t recule = pas > 127;
    if (recule) pas = 0 - pas;
    poser(colonne, ligne, tuile);
    while (pas > 0) {
        for (uint8_t i = 0; i < 15; i++) image();
        if (recule && colonne == 0) break;
        if (!recule && colonne == 19) break;
        effacer(colonne, ligne, 1);
        if (recule) colonne--;
        else colonne++;
        poser(colonne, ligne, tuile);
        pas--;
    }
    return colonne;
}
```

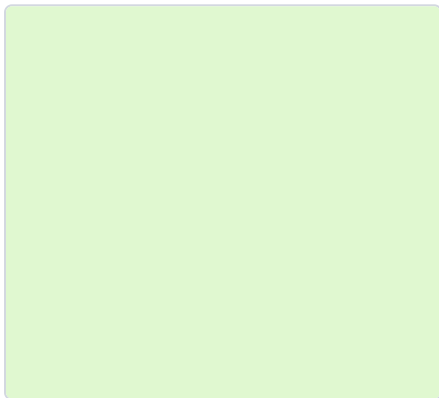
4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    x = mon_deplace_x(x, 0, ALPHABET[0], 5);
    x = mon_deplace_x(x, 0, ALPHABET[0], -5);    // le A (le 0.33)
    xb = mon_deplace_x(xb, 8, ALPHABET[1], 5);
    xb = mon_deplace_x(xb, 8, ALPHABET[1], -5);  // le B (le 0.33.1)

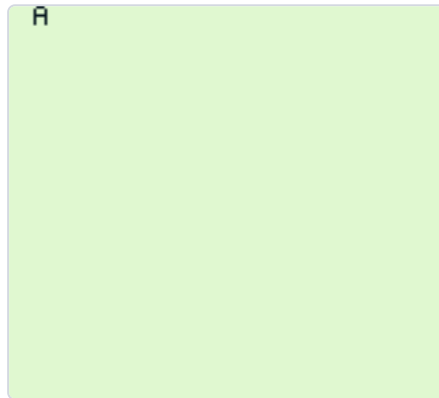
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

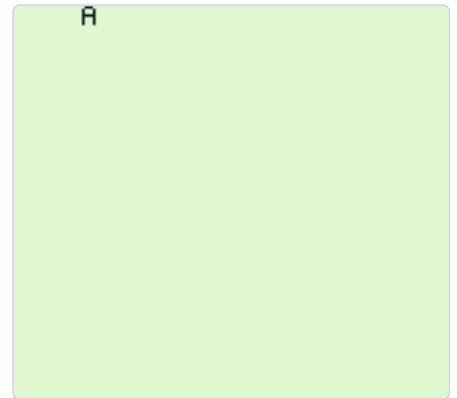
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



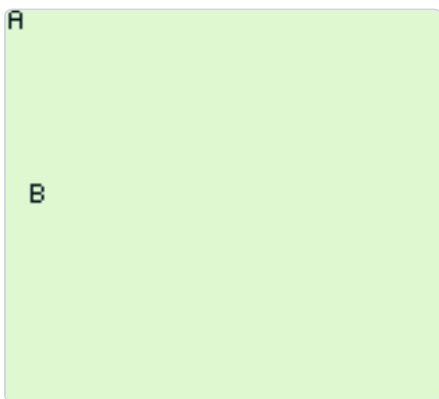
la toute première image — la console vient de commencer — image 1



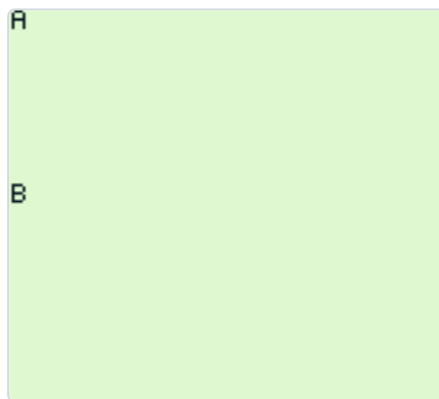
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le A va et revient sur la ligne 0, puis le B sur la ligne 8 : la même fonction pour les deux.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A finit en (0, 0) (0,0)
- ✓ le B passe par (5, 8)
- ✓ le B finit en (0, 8) (0,8)

Ensuite : **0.34. Un carré autour d'une lettre, avec carre** — niveau 0 sur 10. · `gameboy3` — écrire une cartouche Game Boy en C++